PARALLELIZZAZIONE DI UN ALGORITMO PER LA GENERAZIONE DI HEATMAP

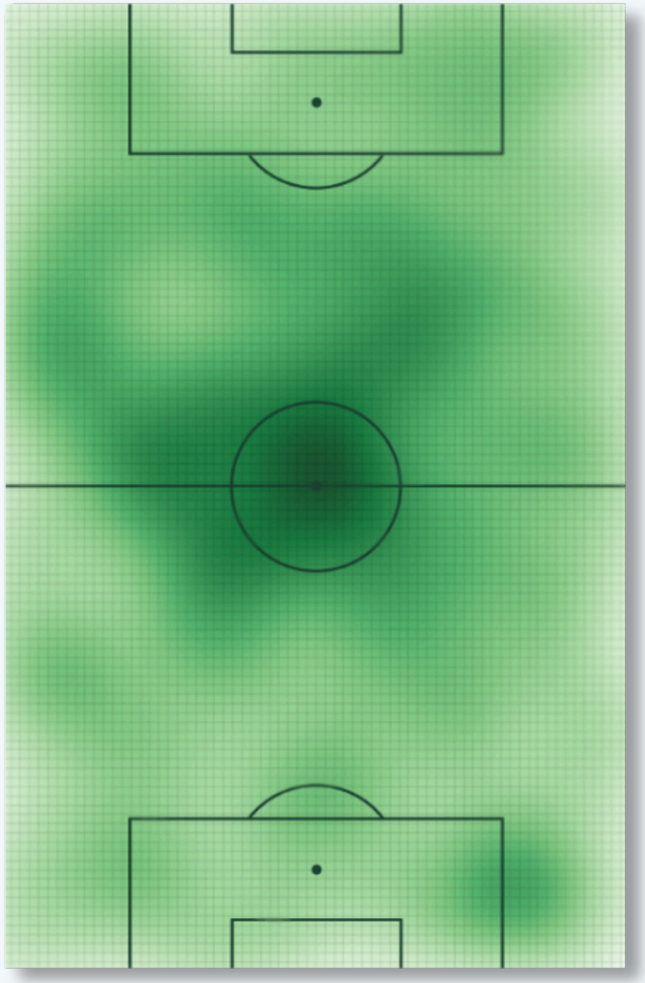
Paolo Savino
Alessandra Tolomelli

Corso di Sistemi di Elaborazione
Accelerata M a.a 2025/2026

Perchè una heatmap?

- Permette di visualizzare la **densità** di milioni di eventi in un'unica immagine intuitiva, rendendo immediatamente comprensibili pattern spaziali complessi.

- Evidenzia non solo la posizione, ma il "**peso**" o l'influenza di un evento su un'area circostante (tramite kernel di densità come quello Gaussiano).



Parallelizzazione dell'algoritmo

Indipendenza del dominio

- Il calcolo del valore di intensità di un singolo pixel *(x,y)* è indipendente dai risultati degli altri pixel
- Non esiste un ordine prestabilito per il calcolo: i pixel possono essere processati in qualsiasi ordine o contemporaneamente

Struttura estremamente parallelizzabile

- Il problema complessivo di generare una heatmap può essere scomposto efficientemente in micro-compiti identici e **indipendenti**

Elevata intensità aritmetica

- La ripetizione della stessa operazione matematica (Kernel di densità) su grandi volumi di dati rende l'algoritmo ideale per architetture che eseguono **istruzioni identiche su flussi di dati multipli**

Implementazione sequenziale Naïve

Inversione della logica: ogni pixel della heatmap agisce come un'unità di lavoro indipendente: interroga il dataset degli eventi e calcola il proprio valore

Zero conflitti di memoria: ogni locazione della heatmap viene scritta una sola volta, eliminando la necessità di sincronizzazione tra le operazioni.

Utilizzo della funzione exp: permette di trasformare una distanza netta in un valore di intensità che decade dolcemente. Più un pixel è lontano dall'evento, minore sarà il suo valore, garantendo tramite l'utilizzo di *inv_sigma* una transizione fluida e non a "gradini".

```
const float radius_sq = 400.0f;
const float inv_sigma = 1.0f / 50.0f;

// loop sui PIXEL (pixel-driven)
for (int y = 0; y < height; ++y){
    for (int x = 0; x < width; ++x){
        float intensity = 0.0f;
        // accumula contributi di TUTTI gli eventi
        for (int i = 0; i < n_events; ++i){
            float dx = (float)x - x_events[i];
            float dy = (float)y - y_events[i];
            float dist_sq = dx * dx + dy * dy;

            if (dist_sq < radius_sq)
            {
                intensity += std::exp(-dist_sq * inv_sigma);
            }
        }
        // scrittura finale (una sola volta)
        heatmap[y * width + x] = intensity;
    }
}
```

CUDA: versioni realizzate

(Tesla T4, CUDA Cores: 2560), Colab

1. Prima implementazione parallela
2. Griglia doppia
3. Spatial partitioning (SMEM): griglia x2
4. Spatial partitioning (SMEM): griglia x128
5. Spatial partitioning (SMEM + ILP): griglia x2
6. Spatial partitioning (SMEM): griglia x32
7. Spatial partitioning (SMEM + ILP): griglia x64

Prima implementazione parallela

Parametri

- w, h = 105, 68
- threads = (16, 16)
- blocks = ((w + threads[0] - 1) // threads[0],
  (h + threads[1] - 1) // threads[1])

Compute (SM) Throughput [%]	35,99
Memory Throughput [%]	27,66
L1/TEX Cache Throughput [%]	33,25
L2 Cache Throughput [%]	0,22
DRAM Throughput [%]	0,04
Theoretical Occupancy [%]	100
Theoretical Active Warps per SM [warp]	32
Achieved Occupancy [%]	21,77
Achieved Active Warps Per SM [warp]	6,96

Speedup: 226x

Sottoutilizzo degli Streaming Multiprocessor: viene utilizzato solo il 20% degli SM perché la griglia di blocchi lanciata è troppo piccola per "coprire" l'intero hardware parallelizzabile, motivo per il quale quasi ogni richiesta di dati viene soddisfatta istantaneamente dalle cache, riducendo al minimo il traffico verso la DRAM.

Compute bound: la larghezza di banda della memoria non rappresenta un collo di bottiglia: la GPU non è in attesa di dati dalla DRAM, ma è limitata esclusivamente dalla velocità di esecuzione delle istruzioni matematiche da parte delle unità di calcolo.

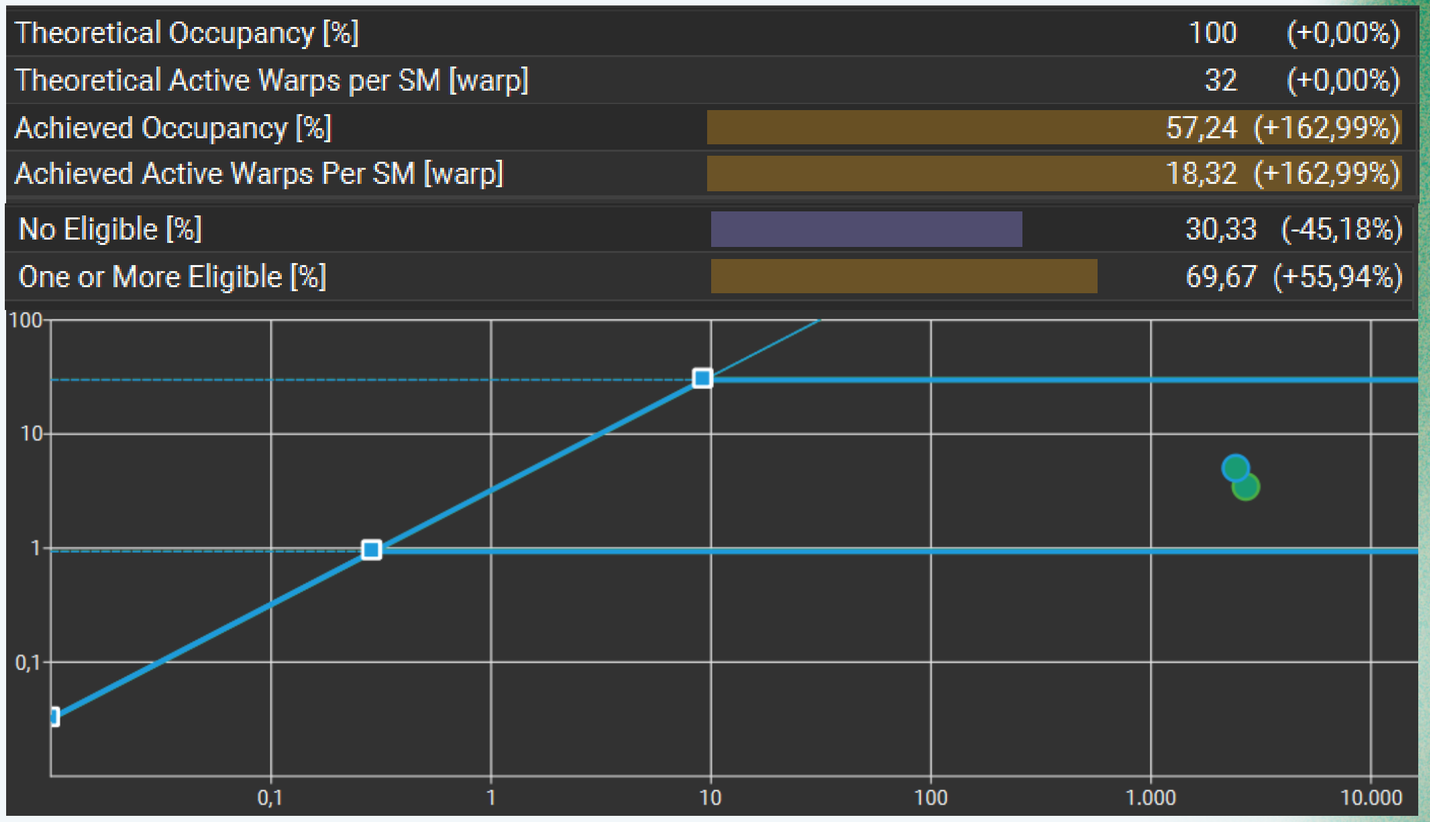
Griglia doppia

Speedup: 294x

Compute Throughput: salito al 63%, confermando che la GPU è ora saturata correttamente.

Sebbene l'**Occupancy** reale sia del 57%, il 30% di stalli rilevato è da considerarsi un valore ottimale. Il sistema dimostra un buon Latency Hiding: la GPU riesce a "nascondere" i tempi morti del calcolo matematico alternando l'esecuzione tra i numerosi warp eleggibili.

Il kernel opera in regime di **Compute Bound**, indicando che il prossimo salto prestazionale non deriverà dalla gestione della memoria, ma dalla densità del calcolo per singolo thread.



105x68 210 x 136

Spatial partitioning

Lo **Spatial Partitioning** è risultato utile per trasformare un ammasso disordinato di 1 milione di eventi in una struttura organizzata, in modo da consentirne una lettura efficiente da parte della GPU.

Assegnazione Spaziale: ogni evento viene mappato su una griglia bidimensionale dividendo le sue coordinate per la *cell_size*. Viene calcolato un *cell_id* univoco per ogni evento, identificando esattamente in quale cella logica ricade.

Ordinamento per Località: gli eventi vengono riordinati in memoria in base al loro *cell_id.* Questo garantisce che quelli appartenenti alla stessa cella siano memorizzati in posizioni contigue, massimizzando l'efficienza della cache durante l'esecuzione del kernel CUDA.

```
cell_size = 20.0
grid_w = int(np.ceil(w / cell_size))
grid_h = int(np.ceil(h / cell_size))

# assegnazione eventi alle celle
cell_x = cp.clip((x_gpu / cell_size).astype(cp.int32), 0, grid_w - 1)
cell_y = cp.clip((y_gpu / cell_size).astype(cp.int32), 0, grid_h - 1)
cell_ids = cell_y * grid_w + cell_x

# Ordina eventi per cella
order = cp.argsort(cell_ids)
x_sorted = x_gpu[order]
y_sorted = y_gpu[order]
cell_ids_sorted = cell_ids[order]

# Inizio / fine eventi per cella
cell_starts = cp.full(n_cells, -1, dtype=cp.int32)
cell_ends   = cp.full(n_cells, -1, dtype=cp.int32)

unique_cells, start_idx = cp.unique(cell_ids_sorted, return_index=True)
counts = cp.bincount(cell_ids_sorted, minlength=n_cells)

cell_starts[unique_cells] = start_idx.astype(cp.int32)
cell_ends[unique_cells]   = (start_idx + counts[unique_cells]).astype(cp.int32)
```

Creazione degli Indici: vengono generati due array ausiliari: *cell_starts* e *cell_ends*, i quali fungono da "mappa" per il kernel, indicando per ogni cella dove inizia e dove finisce la lista dei suoi eventi all'interno del dataset ordinato.

Spatial partitioning: shared memory

Senza **Shared Memory** ogni thread dovrebbe leggere gli eventi direttamente dalla memoria globale, la cui banda passante verrebbe saturata inutilmente da dati identici letti ripetutamente.

Caricamento Cooperativo: tutti i thread del blocco collaborano per caricare gli eventi della cella dalla Global Memory alla Shared Memory.

Efficienza Massima: Ogni evento viene letto una sola volta dalla memoria globale, utilizzando poi *__syncthreads() abbiamo la certezza* che nessun thread inizi il calcolo finché l'intero "set" di eventi non è stato caricato correttamente in Shared Memory.

```
extern __shared__ float shmem[];
float* sh_x = shmem;
float* sh_y = shmem + MAX_EVENTS;

// Caricamento cooperativo
int tid = threadIdx.y * blockDim.x + threadIdx.x;
int n_threads = blockDim.x * blockDim.y;

for (int i = tid; i < n_events; i += n_threads) {
    sh_x[i] = x_events[start + i];
    sh_y[i] = y_events[start + i];
}

__syncthreads();
```

Uso Intensivo della Shared Memory: gli stessi eventi in SMEM vengono riutilizzati istantaneamente da tutti i thread per il calcolo della funzione __expf.

Spatial partitioning: x2 vs x128

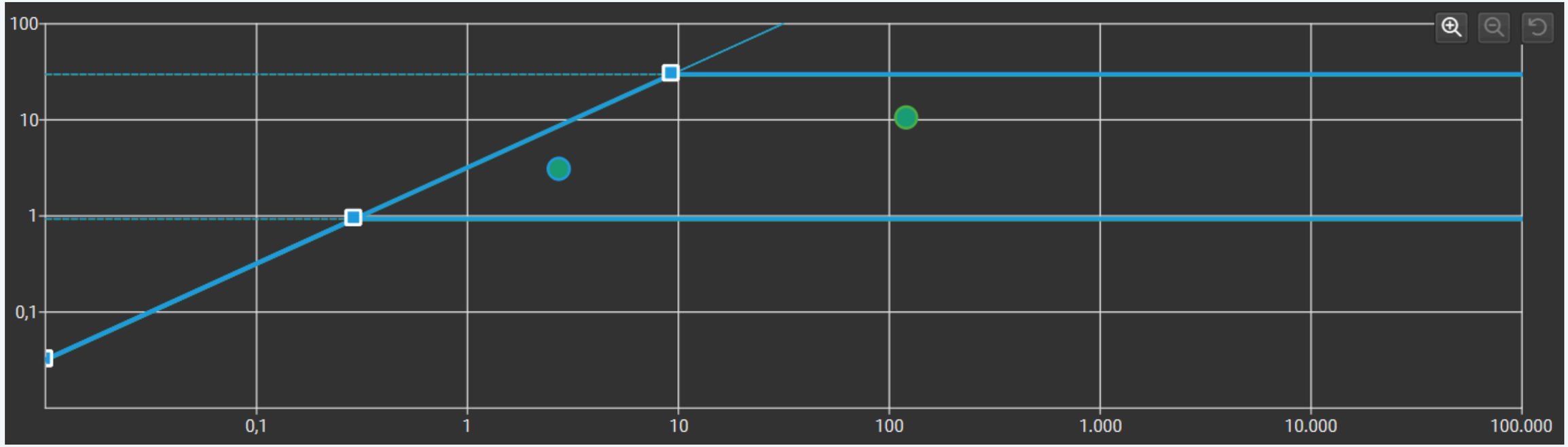
Speedup (x2): 11015x

3 stalli strutturali dominanti:

- loop non uniformi
- dipendenze matematiche
- spill in local memory

Latency Bound →

Theoretical Occupancy [%]	100	(+0,00%)
Theoretical Active Warps per SM [warp]	32	(+0,00%)
Achieved Occupancy [%]	90,70	(+121,65%)
Achieved Active Warps Per SM [warp]	29,02	(+121,65%)
Active Warps Per Scheduler [warp]	7,19	(+117,54%)
Eligible Warps Per Scheduler [warp]	0,72	(-39,75%)
Issued Warp Per Scheduler	0,40	(-27,28%)
No Eligible [%]	59,66	(+33,99%)



Spatial partitioning + ILP

Mapping pixel più fine: Ogni thread gestisce direttamente un pixel, non una cella intera, in questo modo i warp hanno loop di dimensione più uniforme.

Densità binaria: evita la *__expf*, che è molto costosa riducendo stalli da dipendenze matematiche.

Loop unrolling manuale + accumulo in **registri separati** (a0–a3): riducendo la catena di dipendenze è migliorato il modo in cui il warp gestisce le attese in modo da “riempire i buchi” di inattività.

```
for (int i = 0; i < n_events; i += 4) {
    #pragma unroll 4
    for (int k = 0; k < 4; k++) {
        int idx = i + k;
        if (idx < n_events) {
            float dx = px - sh_x[idx];
            float dy = py - sh_y[idx];
            float d2 = dx*dx + dy*dy;

            // aggiunta di densità uniforme
            if (d2 < radius_sq) {
                if (k == 0) a0 += 1.0f;
                else if (k == 1) a1 += 1.0f;
                else if (k == 2) a2 += 1.0f;
                else a3 += 1.0f;
            }
```

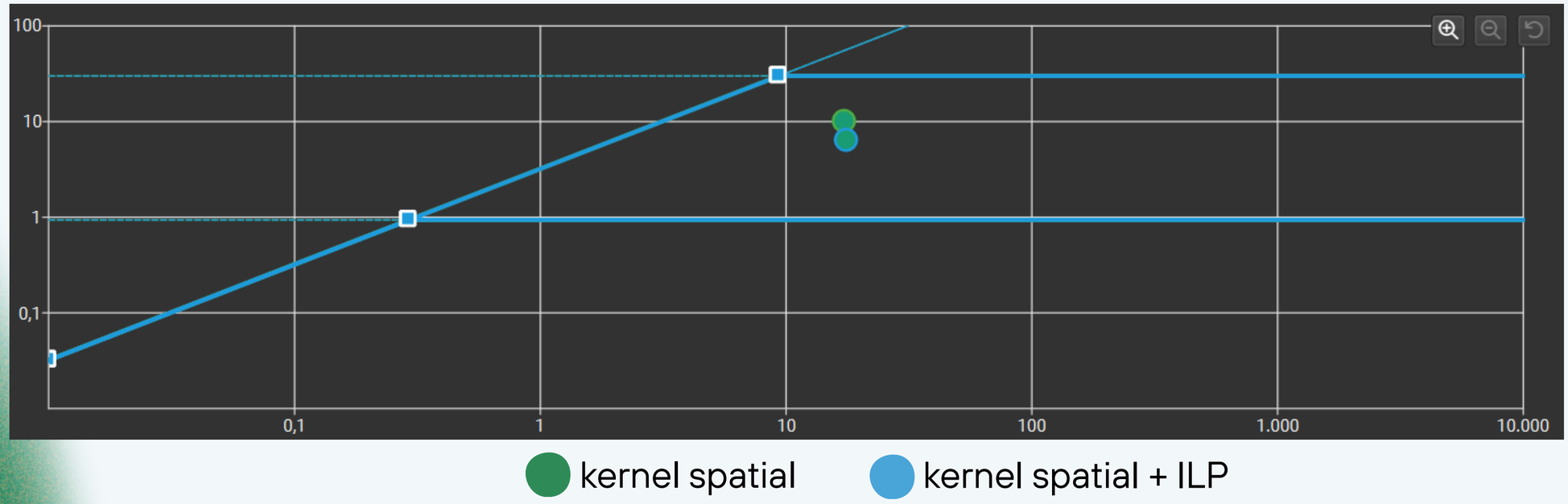
Analisi comparativa delle prestazioni

Confronto basato sulle migliori configurazioni di griglia e parametri di lancio identificati tramite Nsight Compute

Riduzione della risoluzione: migliore bilanciamento tra computazione e accessi in memoria

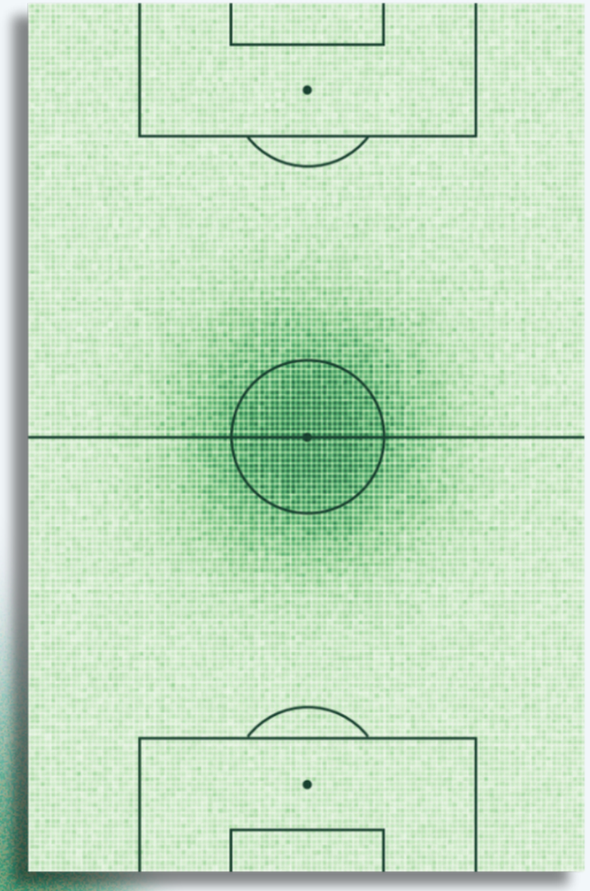
- Kernel Spatial: scale 32
- Kernel Spatial + ILP: scale 64

Dal punto di vista del **Roofline Model** notiamo un'alterazione dell'intensità aritmetica effettiva del carico globale e un conseguente spostamento dell'esecuzione verso il **ridge point**, mettendo in evidenza come la scelta della risoluzione del problema sia un fattore critico quanto l'ottimizzazione del kernel stesso.



Analisi comparativa delle prestazioni

Confronto basato sulle migliori configurazioni di griglia e parametri di lancio identificati tramite Nsight Compute



Compute (SM) Throughput [%]	74,61	(+22,76%)
Memory Throughput [%]	69,06	(+199,65%)
L1/TEX Cache Throughput [%]	77,17	(+83,73%)
L2 Cache Throughput [%]	5,08	(+48,57%)
DRAM Throughput [%]	10,96	(-37,60%)
Duration [ms]	4,52	(+651,88%)

Active Warps Per Scheduler [warp]	7,22	(+24,40%)
Eligible Warps Per Scheduler [warp]	1,70	(-23,85%)
Issued Warp Per Scheduler	0,75	(+18,28%)
No Eligible [%]	25,22	(-31,42%)

Achieved Occupancy [%]	90,65	(+32,77%)

Test su Dataset multipli: i test condotti su diversi volumi di dati indicano che la configurazione a 2 milioni di eventi rappresenta il punto di massima efficienza. Tale rendimento si mantiene costante fino a 50 milioni di eventi, limite massimo testato.

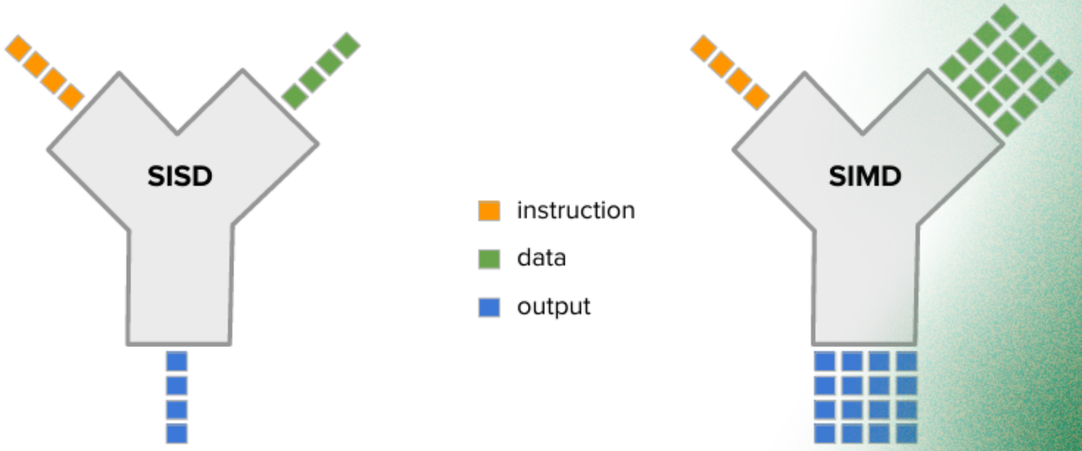
Tuttavia, al di sotto dei 2 milioni di eventi, si riscontra un crollo delle prestazioni dovuto a un utilizzo inefficiente della gerarchia di memoria.

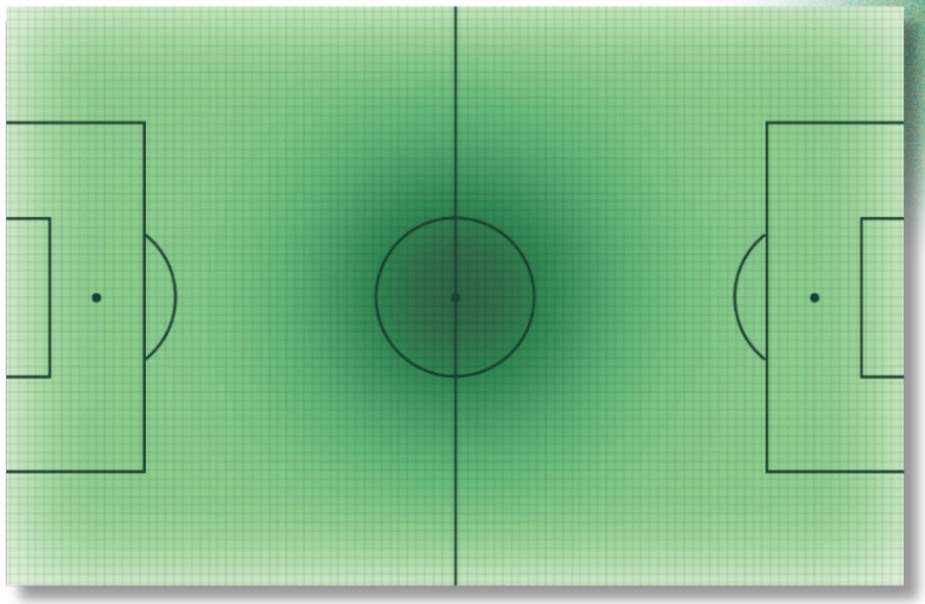
Ulteriori analisi: ottimizzazione SIMD

SIMD è una tecnologia presente nei processori moderni che permette di gestire i dati non più come singoli elementi isolati, ma come vettori.

Ogni volta che la CPU esegue un'istruzione di somma o moltiplicazione, non la applica a un solo evento, ma a un blocco di 8 eventi contemporaneamente, sfruttando appieno i registri a 256 bit del processore, messi a disposizione dall'estensione **AVX**.

Utilizza istruzioni di caricamento vettoriale (*_mm256_loadu_ps*) tramite le quali riempie un intero registro da 256 bit con un singolo fetch dalla cache, con risultante riduzione dei cicli di attesa della CPU per il recupero dei dati degli eventi.





Ulteriori analisi: ottimizzazione SIMD

Calcolo della distanza vettoriale: il calcolo di dy^2 + dx^2 viene effettuato in un'unica operazione hardware, migliorando precisione e velocità (FMA).

Filtraggio tramite mascheramento: viene generata una maschera binaria per verificare quali degli 8 eventi si trovano entro il *radius_sq.*

Gestione della coda: un ciclo scalare classico gestisce gli ultimi eventi rimanenti (se n_events non è un multiplo di 8), garantendo la correttezza totale del risultato.

```
// 8 eventi alla volta
for (; i <= n_events - 8; i += 8)
{
    __m256 ex = _mm256_loadu_ps(&x_events[i]);
    __m256 ey = _mm256_loadu_ps(&y_events[i]);

    __m256 dx = _mm256_sub_ps(vx, ex);
    __m256 dy = _mm256_sub_ps(vy, ey);

    __m256 dist_sq =
        _mm256_fmadd_ps(dy, dy, _mm256_mul_ps(dx, dx));

    // maschera: dist_sq < radius_sq
    __m256 mask = _mm256_cmp_ps(dist_sq, v_radius, _CMP_LT_OQ);

    // scarichiamo solo i valori validi
    float dist_arr[8];
    int mask_bits = _mm256_movemask_ps(mask);
```

Speedup: 2.5x

lo speedup risulta quasi nullo su griglie di piccole dimensioni, iniziando a diventare significativo solo con un incremento del fattore di risoluzione pari a 4x. Nonostante ciò, dal momento che il calcolo per evento include operazioni costose e non vettorializzabili, queste diventano un collo di bottiglia, limitando lo speed-up effettivo ottenibile.

Conclusioni

Il progetto ha confermato che il calcolo delle Heatmap è un problema **estremamente parallelizzabile**, permettendo di ottenere miglioramenti prestazionali drastici grazie al passaggio dall'esecuzione sequenziale a quella parallela.

L'implementazione su architettura CUDA rappresenta il cuore del progetto: grazie alla capacità della GPU di gestire migliaia di thread simultaneamente, è stato possibile abbattere i tempi di calcolo, rendendo la generazione della heatmap quasi istantanea anche su dataset massivi. Il vantaggio competitivo della GPU è stato riscontrato in presenza di un corretto equilibrio tra carico computazionale, parallelismo e banda di memoria. In tali condizioni, la GPU riesce a mantenere una latenza di calcolo molto bassa rispetto alle architetture CPU.

Come termine di paragone, è stata sviluppata un'ulteriore versione ottimizzata su CPU tramite istruzioni SIMD. Sebbene questo approccio abbia permesso di velocizzare il codice sequenziale, esso ne ha evidenziato i **limiti fisici**: oltre una certa risoluzione il tempo di calcolo su CPU diventa eccessivo.